

OPERATING SYSTEMS

Laboratory Manual

Lab 07

Named Pipes & Inter-Process Communication

■ Topic	Named Pipes, Unnamed Pipes, IPC	■ Duration	~ 45 Minutes
■ Environment	Linux / Unix (GCC required)	■ Files	producer.c • consumer.c
■ Level	Intermediate	■ Key Calls	mkfifo • open • read • write

Learning Objectives
✓ Understand how named and unnamed pipes work in Linux/Unix ✓ Implement basic IPC using mkfifo and related system calls ✓ Demonstrate Producer-Consumer communication via named pipes ✓ Handle blocking and non-blocking I/O on pipe file descriptors ✓ Manage proper cleanup of FIFO files after use

Section 1: Introduction to Pipes

1.1 What is a Pipe?

Pipelines are the oldest form of UNIX inter-process communication (IPC). They establish a unidirectional communication channel between processes. A pipe follows the **First In, First Out (FIFO)** principle — data written first is read first. The kernel buffers the data until the reader consumes it, providing a clean handoff between producer and consumer processes.

■ **Note** Pipes strictly follow IPC semantics: one end writes, the other reads. The kernel guarantees atomicity for writes $\leq$ PIPE_BUF bytes (typically 4 096 B).

Key Characteristics

- **Unidirectional** — data flows in one direction by default.
- **FIFO ordering** — first byte written is the first byte read.
- **Kernel-buffered** — data is temporarily held in kernel memory.
- **Blocking by default** — reads block when empty; writes block when full.

Section 2: Unnamed (Ordinary) Pipes

2.1 Overview

An **ordinary (unnamed) pipe** is a transient communication channel created with the **pipe()** system call. It exists only in memory and is accessible only by the creating process and its descendants. Once all process file descriptors referencing the pipe are closed, the pipe is destroyed automatically.

Properties

Property	Detail
Relationship	Related processes only (parent $\leftrightarrow$ child)
Persistence	Lives only as long as the referencing processes
Access	Via file descriptors — not visible in filesystem
Direction	Unidirectional (use two pipes for bidirectional)
System Call	<code>pipe(int fd[2])</code> — <code>fd[0]</code> = read end, <code>fd[1]</code> = write end

Shell Example

The vertical bar `|` in a shell command creates an unnamed pipe between two processes at runtime:

```
# List all processes and filter for root-owned ones

ps aux | grep root

# ps aux writes its output → pipe → grep root reads from pipe
```

C Implementation — Parent → Child

The following skeleton demonstrates how a parent process writes a message to an unnamed pipe and the child process reads it:

```
#include

#include

#include

int main() {

    int fd[2]; // fd[0]=read, fd[1]=write
    pipe(fd); // create the pipe

    if (fork() == 0) { // CHILD
        close(fd[1]); // close unused write end
        char buf[128];

        read(fd[0], buf, sizeof(buf));
        printf("Child received: %s\n", buf);
        close(fd[0]);
    } else { // PARENT
        close(fd[0]); // close unused read end
        char *msg = "Hello from parent";
        write(fd[1], msg, strlen(msg)+1);
        close(fd[1]); // signals EOF to child

        wait(NULL);
    }

    return 0;
}
```

■ Tip

Always close the unused end of the pipe in each process. Failing to close `fd[1]` in the child prevents the parent's EOF from ever being detected.

Section 3: Named Pipes (FIFOs)

3.1 What is a Named Pipe?

A **named pipe** (also called a **FIFO** — First In, First Out) is a special file in the Linux filesystem that acts as a pipe but with a name. Unlike ordinary pipes, a FIFO persists in the filesystem and can be used for communication between **any two unrelated processes** as long as they have the correct file permissions.

Properties

Property	Detail
Creation	mkfifo <name> or mkfifo(path, mode) in C
Location	Exists as a special file in the filesystem (type p)
Relationship	Any two processes — no parent-child requirement
Persistence	Survives until explicitly deleted (rm / unlink)
Direction	Unidirectional per open(); bidirectional needs two FIFOs
Permissions	Standard Unix read/write/execute bits apply

3.2 Creating & Using a Named Pipe (Shell)

```
# Step 1 - Create the named pipe
mkfifo /tmp/my_pipe

# Step 2 - Terminal 1: Write data into the pipe (blocks until reader)
echo "Hello from writer" > /tmp/my_pipe

# Step 3 - Terminal 2: Read from the pipe
cat < /tmp/my_pipe

# Output: Hello from writer

# Step 4 - Verify file type (look for "p" at start of permissions)
ls -l /tmp/my_pipe

# prw-r--r-- 1 user group 0 Oct 2 12:21 /tmp/my_pipe

# Step 5 - Clean up
```

```
rm /tmp/my_pipe
```

■ **Note**

The "p" at the start of the ls -l output identifies the file as a pipe (FIFO). The write in Terminal 1 will **block** until Terminal 2 opens the pipe for reading.

3.3 Named Pipe in C — mkfifo()

System call signature: `int mkfifo(const char *pathname, mode_t mode);`

Returns 0 on success, -1 on error. Requires `#include <sys/stat.h>`.

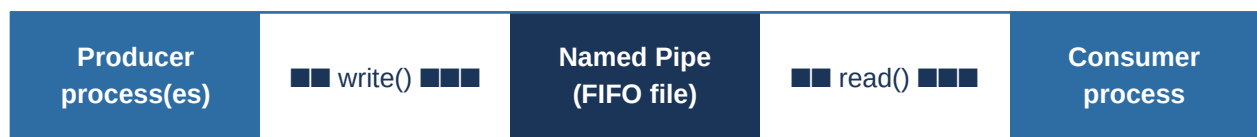
```
return 0;  
}
```

Section 4: Producer-Consumer via Named Pipes

4.1 Concept

The **Producer-Consumer** pattern is a classic concurrency model: one or more **producers** generate data items and place them in a shared buffer; one or more **consumers** remove items and process them. A named pipe acts as the shared buffer — the kernel handles synchronisation automatically through its blocking semantics.

Flow Diagram



4.2 Producer Code (producer.c)

```

#include
#include
#include
#include
#include
#include

#define FIFO_PATH "/tmp/prod_cons_fifo"
#define MSG_COUNT 5

int main() {
    // Create the FIFO (ignore error if it already exists)
    mkfifo(FIFO_PATH, 0666);

    printf("Producer: opening FIFO for writing...\n");
    int fd = open(FIFO_PATH, O_WRONLY);
    if (fd == -1) { perror("open"); exit(EXIT_FAILURE); }

    for (int i = 1; i <= MSG_COUNT; i++) {
        char msg[64];
  
```

```
snprintf(msg, sizeof(msg), "Item %d from producer", i);  
write(fd, msg, strlen(msg) + 1);  
printf("Producer sent: %s\n", msg);  
sleep(1); // simulate work  
}  
  
close(fd);  
printf("Producer: done.\n");  
return 0;  
}
```

4.3 Consumer Code (consumer.c)

```
#include  
  
#include  
  
#include  
  
#include  
  
#include  
  
#define FIFO_PATH "/tmp/prod_cons_fifo"  
  
int main() {  
    printf("Consumer: opening FIFO for reading...\n");  
    int fd = open(FIFO_PATH, O_RDONLY);  
    if (fd == -1) { perror("open"); exit(EXIT_FAILURE); }  
  
    char buf[256];  
    int n;  
    while ((n = read(fd, buf, sizeof(buf))) > 0) {  
        printf("Consumer received: %s\n", buf);  
    }  
  
    close(fd);  
    unlink(FIFO_PATH); // clean up the FIFO  
    printf("Consumer: pipe closed and cleaned up.\n");  
    return 0;  
}
```

```
}
```

```
# Compile both programs  
  
gcc -o producer producer.c  
  
gcc -o consumer consumer.c  
  
# Terminal 1 – start consumer first (it blocks waiting for producer)  
  
./consumer  
  
# Terminal 2 – run producer  
  
./producer
```

■ Tip

Start the **consumer before the producer**. The consumer's `open()` call blocks until a writer opens the other end — this is the kernel's built-in synchronisation at work.

Section 5: Named vs. Unnamed Pipes — Key Differences

Aspect	Unnamed Pipe	Named Pipe (FIFO)
Creation	pipe(fd)	mkfifo() / mkfifo shell cmd
Filesystem entry	No	Yes — appears as type "p"
Process scope	Related processes only	Any processes with permission
Persistence	Destroyed on process exit	Persists until unlink()/rm
IPC scope	Within a process tree	System-wide (same machine)
Typical use	Shell pipelines, parent↔child	Unrelated daemon ↔ client
Cleanup	Automatic	Manual (unlink required)

Section 6: Blocking Behaviour & Non-Blocking Mode

6.1 Default Blocking Rules

By default every pipe operation is **blocking**. The kernel pauses the calling process until the condition it requires is met.

Scenario	What Happens	Unblocks When
Read — empty pipe	Reader process sleeps	Writer puts data in pipe
Read — no writer open	Reader gets EOF (0 bytes returned)	N/A — pipe is done
Write — pipe full	Writer process sleeps	Reader consumes data
Write — no reader open	SIGPIPE signal + EPIPE errno	N/A — broken pipe
open(FIFO, O_RDONLY)	Blocks until a writer opens other end	Writer calls open(O_WRONLY)
open(FIFO, O_WRONLY)	Blocks until a reader opens other end	Reader calls open(O_RDONLY)

6.2 Non-Blocking Read Example

Pass **O_NONBLOCK** to open() or use **fcntl()** to switch an already-open descriptor to non-blocking mode. The call then returns immediately with errno set to **EAGAIN** if no data is available.

```
#include
```

```
#include

#include

#include

#include

int main() {
    const char *fifo = "/tmp/my_fifo";
    char buf[100];

    // O_NONBLOCK: open returns immediately even if no writer
    int fd = open(fifo, O_RDONLY | O_NONBLOCK);
    if (fd == -1) { perror("open"); exit(EXIT_FAILURE); }

    while (1) {
        int n = read(fd, buf, sizeof(buf));
        if (n > 0) {
            printf("Read: %s\n", buf);
        } else if (n == -1 && errno == EAGAIN) {
            printf("No data yet – will retry...\n");
        } else {
            printf("EOF or pipe closed.\n");
            break;
        }
        sleep(1);
    }
    close(fd);
    return 0;
}
```

6.3 Non-Blocking Write Example

```
#include

#include

#include

#include
```

```
#include

#include

int main() {
    const char *fifo = "/tmp/my_fifo";
    const char msg[] = "Hello from producer";

    int fd = open(fifo, O_WRONLY | O_NONBLOCK);
    if (fd == -1) {
        perror("Error opening FIFO (no reader?)");
        exit(EXIT_FAILURE);
    }

    if (write(fd, msg, strlen(msg)+1) == -1) {
        perror("Write failed");
    } else {
        printf("Message sent: %s\n", msg);
    }

    close(fd);
    return 0;
}
```

**Warning**

With `O_NONBLOCK` on a named pipe, `open(O_WRONLY)` returns `ENXIO` if no reader has the pipe open. Always check the return value and `errno`.

6.4 Handling EOF

When the last writer closes the pipe the reader receives an **EOF** (`read()` returns 0). The reader should always check for this condition and exit its read loop cleanly:

```
while ((n = read(fd, buf, sizeof(buf))) > 0) {
    // process buf[0..n-1]
}

if (n == 0) printf("EOF: all writers closed the pipe.\n");
if (n == -1) perror("read error");
```

Section 7: System Calls Quick Reference

Call / Command	Signature / Usage	Purpose
pipe()	int pipe(int fd[2]);	Create unnamed pipe
mkfifo() [C]	int mkfifo(const char *path, mode_t mode);	Create named pipe file
mkfifo [shell]	mkfifo <filename>	Shell command to create FIFO
open()	int open(const char *path, int flags);	Open file/pipe for I/O
read()	ssize_t read(int fd, void *buf, size_t count);	Read bytes from pipe
write()	ssize_t write(int fd, const void *buf, size_t count);	Write bytes to pipe
close()	int close(int fd);	Close file descriptor
unlink()	int unlink(const char *path);	Delete FIFO file
fcntl()	int fcntl(int fd, int cmd, ...);	Set O_NONBLOCK on fd
fork()	pid_t fork(void);	Create child process
wait()	pid_t wait(int *status);	Wait for child exit

Section 8: Lab Tasks

Complete the following five tasks in order. Each builds on knowledge from the previous sections. Read the hints and important points before writing code.

Task 1 Named Pipe — Send, Receive & Cleanup

Description	Write a C program that: (a) creates a named pipe using <code>mkfifo()</code> , (b) forks a child process — child reads a message, parent writes it, (c) both sides close their descriptors after use, and (d) the parent calls <code>unlink()</code> to remove the FIFO file before exiting.
Est. Time	~5 minutes
Objectives	• Use <code>mkfifo("/tmp/task1_fifo", 0666)</code> to create the FIFO. • <code>fork()</code> returns 0 in child and the child's PID in parent. • Child opens FIFO with <code>O_RDONLY</code>; parent opens with <code>O_WRONLY</code>. • Parent must call <code>wait(NULL)</code> to avoid zombie processes. • Verify the FIFO is deleted: <code>ls -l /tmp/task1_fifo</code> should fail.
Hints	• Call <code>mkfifo()</code> before <code>fork()</code> so both processes see the same file. • Use <code>strlen(msg)+1</code> to also transmit the null terminator. • If <code>open()</code> hangs, make sure both ends are eventually opened.

■ **Note** **Important Points:** A FIFO `open()` blocks until both a reader AND writer have opened the pipe. Calling `unlink()` is mandatory — orphaned FIFOs consume inode entries and confuse subsequent runs of the same program.

Task 2 Non-Blocking Read with Graceful Error Handling

Description	Modify a named pipe program so the reader uses <code>O_NONBLOCK</code> . If no data is available (<code>EAGAIN</code>), print "No data yet — retrying..." and sleep 1 second before trying again. Exit cleanly on EOF or after 10 retries with no data.
Est. Time	~5 minutes
Objectives	• Pass <code>O_RDONLY O_NONBLOCK</code> to <code>open()</code>. • Check <code>errno == EAGAIN</code> when <code>read()</code> returns <code>-1</code>. • Count retries; exit the loop after 10 failed attempts. • Print the received message when <code>read()</code> returns <code>> 0</code>. • Always <code>close(fd)</code> and optionally <code>unlink()</code> on exit.
Hints	• You can test by running the reader in one terminal before starting the writer. • Use <code>errno.h</code> and include <code>string.h</code> for <code>strerror(errno)</code> in debug messages. • Combine with a producer from Task 1 to observe the full interaction.

■ Note

Important Points: `O_NONBLOCK` converts a blocking wait into a polling loop — efficient only when paired with a sleep or event-driven mechanism. Without `sleep(1)` your loop will spin at 100% CPU. `EAGAIN` and `EWOULDBLOCK` are synonyms on Linux.

Task 3 Multiple Producers → Single Consumer

Description	Implement a system with <code>N=3</code> producer processes each writing 3 messages to a single shared named pipe. A single consumer process reads and prints all messages until it receives EOF from all producers.
Est. Time	~15 minutes
Objectives	• Create the FIFO once before spawning producers (e.g. in <code>main()</code>). • Each producer opens the FIFO, writes its messages, then closes it. • The consumer reads in a loop until <code>read()</code> returns 0 (EOF). • The consumer's EOF arrives only after ALL producers have closed the FIFO. • Use <code>snprintf</code> to tag messages with the producer's PID or index.
Hints	• Fork 3 children; each child is a producer. The parent is the consumer. • Parent should open FIFO for reading after forking all children. • Message interleaving is normal — the kernel handles atomic writes $\leq$ <code>PIPE_BUF</code>. • Call <code>wait()</code> in a loop until all children exit before cleanup.

- **Note** **Important Points:** Named pipes support multiple concurrent writers safely for writes $\leq$ PIPE_BUF (4 096 bytes on Linux). Larger writes may be interleaved. The consumer sees EOF only when every open write descriptor to the FIFO is closed.

Task 4 Parent Writes Multiple Messages; Child Reads Until Empty

Description	Write a program where a parent process writes 5 numbered messages ("Message 1" ... "Message 5") to an unnamed pipe, then closes the write end. The child reads each message one by one and prints it until it receives EOF.
Est. Time	~10 minutes
Objectives	• Use pipe(fd) — NOT mkfifo — for this task (unnamed pipe). • Parent closes fd[0] (read end); child closes fd[1] (write end). • Parent writes all 5 messages, then calls close(fd[1]) to signal EOF. • Child loops: while(read(fd[0], buf, N) > 0) { print buf; } • Parent calls wait(NULL) after all writes.
Hints	• Each message should include its own null terminator (+1 byte). • To read messages individually, either include a length prefix or write fixed-size chunks. • A simpler approach: write one message per write() and read one per read().

- **Note** **Important Points:** Closing the write end of a pipe is *the* signal that tells the reader "no more data is coming." If the parent forgets close(fd[1]), the child will block forever waiting for data that never arrives — a classic deadlock scenario to recognise and avoid.

Task 5 Experimenting with Blocking — Delayed Parent Write

Description	Create a program where the child process immediately tries to read from an unnamed pipe, but the parent delays writing by 5 seconds (<code>sleep(5)</code>). Observe and document the blocking behaviour. Then modify the child to use non-blocking mode and compare the outputs.
Est. Time	~10 minutes
Objectives	• Part A: child calls <code>read()</code> immediately; parent sleeps 5 s then writes. • Part B: child sets <code>O_NONBLOCK</code> on <code>fd[0]</code> using <code>fcntl()</code>; retry loop for 10 s. • Print timestamps (use <code>time()</code>) to show when each event occurs. • Observe console output to confirm child blocks in Part A. • In Part B observe <code>EAGAIN</code> messages before the parent writes.
Hints	• <code>fcntl(fd[0], F_SETFL, O_NONBLOCK)</code> switches an existing fd to non-blocking. • Use <code>time.h</code>: <code>time_t t = time(NULL);</code> to log timestamps. • Add a <code>sleep(1)</code> in the child's retry loop for Part B.

■ Note

Important Points: This task demonstrates the fundamental synchronisation property of pipes: blocking read naturally synchronises the consumer with the producer at zero cost. Non-blocking mode trades this convenience for responsiveness — the process must actively poll and decide what to do during the wait.

Section 9: Summary & Best Practices

9.1 Key Takeaways

Pipes are kernel buffers	Data is stored in kernel memory; no filesystem I/O overhead.
Named pipes are files	mkfifo creates a special "p"-type file; clean up with unlink().
Both ends must be open	A FIFO open() blocks until both reader and writer are connected.
EOF signals end-of-data	The reader gets EOF (n=0) only after ALL writers close the pipe.
Non-blocking = polling	O_NONBLOCK prevents sleeping; add delays to avoid busy-waiting.
PIPE_BUF atomicity	Writes $\leq$ PIPE_BUF bytes are atomic — safe for multiple writers.
Close unused ends	Always close the pipe end you don't use — avoids deadlock/SIGPIPE.

9.2 Common Mistakes

- Not closing unused pipe ends → deadlock or SIGPIPE crash.
- Forgetting unlink() → orphaned FIFO causes "File exists" error on next run.
- Not checking return value of read()/write() → silent data loss.
- Spinning without sleep() in non-blocking mode → 100% CPU usage.
- Writing strings without +1 for null terminator → garbled output.
- Opening FIFO O_WRONLY without a reader → blocks or returns ENXIO (non-blocking).

9.3 Useful Commands

```
# Create a named pipe
mkfifo /tmp/myfifo

# Check file type (p = pipe)
ls -l /tmp/myfifo

# Remove named pipe
rm /tmp/myfifo
```

```
# Compile C programs

gcc -Wall -o writer writer.c
gcc -Wall -o reader reader.c

# Run with timestamp prefix

date; ./consumer &
date; ./producer
```